

Le Protocole HTTP

1. Qu'est-ce que HTTP ?

Le protocole **HTTP** (*HyperText Transfer Protocol*) est le pilier du World Wide Web.

- **Couche Application** : Situé au-dessus de la pile TCP/IP.
- **Modèle Client-Serveur** : Le client fait une demande, le serveur répond.
- **Sans état (Stateless)** : Chaque couple Requête/Réponse est indépendant. Le protocole ne se "souvient" pas des requêtes précédentes (utilisation de cookies/tokens pour y remédier).

2. Le Cycle Requête / Réponse

La communication se déroule toujours en trois étapes clés :

1. **Le Client (ex: Navigateur)** initie la connexion et envoie une **Requête HTTP**.
2. **Le Serveur** reçoit la requête, traite l'information (requête en base de données, logique métier).
3. **Le Serveur** renvoie une **Réponse HTTP** contenant un code de statut et généralement une ressource (HTML, JSON, Image).

3. Les Verbes HTTP (Les Méthodes)

Les verbes indiquent au serveur l'action qu'il doit exécuter sur la ressource ciblée.

- **GET** : Demande une représentation de la ressource (Lecture seule).
- **POST** : Soumet des données pour **créer** une nouvelle ressource.
- **PUT** : **Remplace** intégralement la ressource ciblée par les nouvelles données.
- **PATCH** : Modifie **partiellement** une ressource existante.
- **DELETE** : **Supprime** la ressource spécifiée.

4. Propriétés des Verbes HTTP

Il est important de respecter la sémantique des verbes lors du développement (notamment pour les API REST).

Méthode	Action CRUD	Sécurisée (Sûre)	Idempotente
GET	Read	Oui	Oui
POST	Create	Non	Non
PUT	Update (Total)	Non	Oui
PATCH	Update (Partiel)	Non	Non
DELETE	Delete	Non	Oui

5. Les 5 Catégories de Codes HTTP

Le serveur utilise un code à trois chiffres pour formaliser sa réponse. Le premier chiffre permet de classer immédiatement le résultat :

- **1xx (Informations)** : Requête reçue, le serveur poursuit le traitement (ex: **101 Switching Protocols**).
- **2xx (Succès)** : L'action a été reçue, comprise et acceptée avec succès (ex: **200 OK** , **201 Created**).
- **3xx (Redirection)** : Le client doit accomplir une action supplémentaire pour finaliser la requête (ex: **301 Moved Permanently** , **304 Not Modified**).
- **4xx (Erreur du Client)** : La requête contient une mauvaise syntaxe ou ne peut pas être traitée (ex: **400 Bad Request** , **401 Unauthorized** , **404 Not Found**).
- **5xx (Erreur du Serveur)** : Le serveur a rencontré un problème interne et a échoué à traiter une requête pourtant valide (ex: **500 Internal Server Error** , **502 Bad Gateway**).

6. L'Anatomie d'un Message HTTP

Qu'il s'agisse d'une requête ou d'une réponse, la structure d'un message reste identique et suit un ordre strict :

1. La ligne de départ :

- *Pour une requête* : Spécifie la méthode, l'URL cible et la version du protocole (ex: `GET /api/users HTTP/1.1`).
- *Pour une réponse* : Spécifie la version du protocole, le code de statut et son libellé (ex: `HTTP/1.1 200 OK`).

2. Les En-têtes (Headers) : Des lignes de métadonnées sous la forme `Clé: Valeur` (ex: `Content-Type: application/json` , `Authorization: Bearer <token>`).

3. Une ligne vide : Un séparateur technique obligatoire qui indique au parseur la fin des métadonnées.

4. Le Corps (Body) : Les données brutes (JSON, HTML, fichier). Il est optionnel pour certaines requêtes comme `GET` .